Case beskrivelse – AKR

Om projektet

Projektet AKR er et cozy farming-spil udviklet i Unity Game Engine ved brug af C#. Spillet foregår i en nordisk inspireret verden, hvor spilleren starter som en overlevende og gradvist opbygger et liv gennem farming, dyrepleje og udforskning.

Projektet har fokus på en rolig gameplay-oplevelse med progression gennem farming-baserede systemer frem for stress eller kamp.

Core Gameplay og funktioner

Projektets fokus ligger på følgende gameplay-systemer:

- Dyrkning af afgrøder med vækst over tid
- Dyresystem med interaktion og progression
- Ressourceindsamling og processing
- Exploration af områder og biomes
- Inventory og tool progression
- Day/Night system
- Basic fishing system som ekstra aktivitet
- Simpel building/placement af objekter som fx chests og mindre funktionelle elementer

Gameplay-loopet består af at samle ressourcer, forbedre effektivitet og gentage processen gennem farming og udforskning.

Verdensdesign

Spillet foregår i en nordisk inspireret verden med forskellige biomes såsom skov, kyst og sneområder. Hvert område giver variation i materialer og dyr samt visuel variation i gameplay.

MVP – Minimum Viable Product

Første version af AKR fokuserer på en mindre spilbar prototype med følgende funktioner:

- Ét biome
- To typer afgrøder
- Ét dyresystem
- Growth system
- Soft hunger system
- Inventory system
- Dag/Nat cyklus

MVP'en har fokus på de primære gameplay-mekanikker frem for komplette spilfeatures.

Tekniske mål

Projektet udvikles i Unity med C# og har fokus på:

- Modular kodeopbygning
- Unity C# scripting
- Gameplay systems (farming, inventory, exploration)
- Animationer via Unity Animator
- Performance og struktur

- Iterativ udviklingsproces

Problemformulering – AKR

I en spilbranche hvor mange moderne spil fokuserer på konkurrence, tempo, præstation og konstant progression, oplever flere spillere, at spil i mindre grad fungerer som et afslappende frirum fra hverdagens stress. I stedet designes mange spil omkring pres, optimering og effektivitet, hvilket kan skabe en oplevelse af stress frem for fordybelse.

Hvordan kan vi designe og implementere et pixelart-baseret spil i Unity (AKR), der kombinerer farming, udforskning og ressourcehåndtering i et nordisk inspireret univers, med fokus på en afslappende og fordybende spiloplevelse, samtidig med at spillets systemer opbygges struktureret og modulært, så de er stabile, udvidbare og teknisk veldesignede?

Underspørgsmål

Hvordan kan gameplay-loopet struktureres, så spilleren oplever progression uden tidspres eller strafmekanikker?

Hvordan kan gameplay-mekanikker implementeres i Unity gennem komponentbaseret udvikling, så funktioner som farming og inventory er fleksible under udviklingen?

Hvordan kan en teknisk arkitektur i Unity opbygges, så gameplay-systemer som farming, inventory og time er løst koblede og nemme at udvide?

Hvilke tekniske og designmæssige overvejelser ligger bag valget af Unity som engine til udviklingen af AKR-projektet?

KRAVSPECIFIKATION – AKR

KRAVSPECIFIKATION – AKR 4
1. Indledning 5
1.1 Formål 5
1.2 Referencer 5
1.3 Læsevejledning 5
2. Generel beskrivelse 5
2.1 Systembeskrivelse 5
2.2 Systemets funktion 5
2.3 Systembegrænsninger 5
2.4 Systemets fremtid 6
2.5 Brugerprofil 6
2.6 Løsningsoversigt 6
2.7 Krav til udviklingsforløbet 6
3. Specifikke krav 7
3.1 Funktionelle krav 7
3.2 Informationsmodel 8
3.3 Procesmodel 8
3.4 Løsningsarkitektur 8
3.5 Prioritering (MoSCoW) 8
3.6 Acceptkriterier 9
4. Eksterne grænseflader 10
4.1 Brugergrænseflade 10
4.2 Hardware 10
4.3 Software 10
5. Krav til ydelse 10
6. Kvalitetsfaktorer 10

1. Indledning

1.1 Formål

Formålet med dette projekt er at udvikle spillet AKR, et cozy farming-spil udviklet i Unity, hvor spilleren kan udforske, samle ressourcer og opbygge progression i et nordisk inspireret univers. Produktet skal levere en afslappende spiloplevelse med fokus på modulær opbygning, stabilitet og udvidelsesmuligheder. Systemet designes med henblik på at kunne understøtte én eller flere spillere afhængigt af projektets endelige scope og tidsramme.

1.2 Referencer

- Ønsket om at spillet skal være i 2D kræver en optimeret game engine til den type spil, af denne grund har vi valgt Unity som hoved værktøj frem for Unreal Engine eller andre alternativer, da Unity er bedre udstyret til at behandle denne type af spil.

- Vores valg af programmeringssprog er C#. Grunden til dette er, at Unity er tæt integreret med C#, som bruges til at styre spillets logik, objekter og adfærd. Dette gør sproget til et naturligt valg for os. Derudover ligger sproget tæt op ad vores faglige profil, da skolens undervisningsforløb i høj grad er opbygget omkring C#.

1.3 Læsevejledning

Dokumentet beskriver først systemets overordnede funktion og begrænsninger, derefter en løsningsoversigt samt konkrete funktionelle krav og kvalitetskrav. Dokumentet inkluderer desuden informationsmodel, procesmodel, løsningsarkitektur, prioritering (MoSCoW) og acceptkriterier.

---

2. Generel beskrivelse

2.1 Systembeskrivelse

AKR er et pixelart spil udviklet i Unity med C#. Spilleren interagerer via mus og tastatur og styrer sin progression gennem farming, exploration og ressourcehåndtering. Spillet designes primært omkring en spilleroplevelse, men systemarkitekturen opbygges modulært med henblik på mulig udvidelse til multiplayer.

2.2 Systemets funktion

Spillet skal give spilleren mulighed for at: dyrke afgrøder, samle ressourcer, udforske områder, håndtere inventory og opleve et day/night-system. Spillet skal desuden understøtte

et afgrænset “basic building/placement”, hvor spilleren kan placere simple objekter i verdenen.

2.3 Systembegrænsninger

Spillet fokuserer på afslappende gameplay uden strafmekanikker. Følgende funktioner er udeladt i denne iteration: online services (fx accounts, matchmaking, chat og dedicated server-drift).
[Antagelse: “online features” forstås som online services; ikke lokale/offline systemer.]

Multiplayer: Multiplayer er ikke et baselinekrav i denne iteration og indgår derfor ikke i Must-kravene. Multiplayer kan implementeres som optional funktion, afhængigt af tid og ressourcer (se MoSCoW).

2.4 Systemets fremtid

Systemet designes modulært med henblik på udvidelse og nem tilføjelse af nye mekanismer eller spilprincipper. Mulige fremtidige udvidelser kan omfatte peer-to-peer multiplayer, flere biomes med unikke farver, planter og crops, flere dyr og crops i forskellige varianter samt et mere avanceret byggesystem.

2.5 Brugerprofil

Systemet henvender sig til flere brugergrupper med forskellige forudsætninger og adgangsformer.

Brugergrupper

- **Primær brugergruppe – Spillere:** Casual spillere som ønsker en afslappende spiloplevelse med fokus på farming, exploration og progression.
- **Sekundær brugergruppe – Udviklere:** Udviklere som arbejder videre på projektet i Unity og vedligeholder systemets funktioner og struktur.

Brugernes forudsætninger

- **Spillere** forventes ikke at have teknisk erfaring og skal kunne navigere spillet via intuitive kontroller og UI.
- **Udviklere** forventes at have grundlæggende kendskab til Unity, C# og modulær systemopbygning.

Adgangsform

- Spillere tilgår systemet via spillets brugergrænseflade med mus og tastatur på PC.
- Udviklere tilgår projektet gennem Unity Editor samt versionsstyringssystemet.

2.6 Løsningsoversigt

AKR består af flere gameplay-systemer, som tilsammen udgør spillets kerneoplevelse: farming, inventory, exploration, ressourcehåndtering, day/night og basic building/placement. Systemerne designes modulært, så funktionalitet kan udvikles og udvides uafhængigt.

Spillets gameplay-loop består overordnet af, at spilleren udforsker verdenen og samler ressourcer, anvender værktøjer til farming og basic placement, gemmer ressourcer i inventory og opnår progression gennem arbejde og planlægning.

2.7 Krav til udviklingsforløbet

Projektets udviklingsforløb skal følge faste tekniske og dokumentationsmæssige rammer for at sikre struktur, sporbarhed og kvalitet gennem hele udviklingsperioden.

Udviklingsplatform

- Projektet skal udvikles i Unity Game Engine ved brug af C#.
- Systemerne skal opbygges modulært i overensstemmelse med løsningsarkitekturen.

Versionsstyring

- Versionsstyring skal anvendes via Unitys indbyggede cloud-løsning.
- Ændringer i funktionalitet og nye implementationer skal kunne spores gennem versionshistorik.

Logbog og dokumentation

- Alle væsentlige ændringer og nye features skal dokumenteres i logbogen med korrekt dato og tidspunkt.
- Under udviklingsperioden skal der udarbejdes en teknisk rapport, som beskriver:
    - Spillets tekniske opbygning
    - Systemarkitektur og centrale systemer
    - Centrale scripts og kodeprincipper

Diagramkrav
Projektet skal indeholde følgende dokumentation:

- Kravspecifikation og designbeskrivelse
- Use Cases for centrale gameplay-funktioner
- E/R-diagram for inventory- og datastruktur

---

3. Specifikke krav

3.1 Funktionelle krav

Spillet skal kunne:

Gameplay
- Spilleren skal kunne plante og høste afgrøder.
- Spilleren skal kunne samle ressourcer fra verdenen.
- Spilleren skal kunne udforske områder.

Systemer
- Inventory-system skal håndtere items.
- Day/Night-system skal ændre spillets tilstand over tid.
- Basic building/placement-system skal kunne placere simple objekter.

Brugeroplevelse
- Gameplay-loop må ikke indeholde strafmekanikker.
- Spilleren skal opleve progression gennem arbejde og planlægning.

3.2 Informationsmodel

Systemet baseres på følgende centrale informationsenheder: Player, Inventory, Item, Crop, WorldObject og TimeState.

Player har et Inventory. Inventory indeholder 0..n Items. Crop repræsenterer en plantet afgrøde med vækststadie og høst-output. TimeState repræsenterer tidsprogression (fx dag/nat) og påvirker logik såsom crop-vækst. WorldObject repræsenterer placerbare objekter fra building/placement-systemet.

3.3 Procesmodel

Et generelt flow for interaktion er: Input → handling → systemopdatering → UI feedback.

Farming-flow: Spilleren vælger seed i inventory, planter i verdenen, time/day-night opdaterer vækststadie, spilleren høster, inventory opdateres, og UI viser ændringen.

Resource pickup-flow: Spilleren interagerer med en ressource i verdenen, et item tilføjes til inventory, og UI opdateres.

3.4 Løsningsarkitektur

Løsningen opdeles i løst koblede moduler: Player Controller, Farming System, Inventory System, Time System, Building/Placement System og UI Layer. Modulerne kommunikerer via klare grænseflader, så ændringer i ét system i begrænset omfang påvirker de øvrige.

Optional modul: Networking Module kan tilføjes for multiplayer.
[Antagelse: Valgt netværksmodel er uafklaret (peer-to-peer vs. dedicated).]

3.5 Prioritering (MoSCoW)

MoSCoW-prioriteringen er autoritativ for scope. Den er gjort konsistent med “Systembegrænsninger”, så “Udeladt/Excluded” ikke samtidig optræder som Must/Should/Could.

Feature	**Status**	**Noter**
Farming (plant/høst)	Must	Kerne i cozy farming-loop.
Ressourcesamling	Must	Pickup/Harvest giver items.
Exploration	Must	Udforskning af områder.
Inventory + UI visning	Must	Håndtering og feedback til spiller.
Day/Night-system	Must	Tidsprogression påvirker verden/logik.
Basic building/placement	Must	Afgrænset til simple objekter.
Progression uden strafmekanikker	Must	Designkrav for “cozy” oplevelse.
Multiplayer (netværk)	Could	Optional, afhænger af tid og netværksmodel
Peer-to-peer multiplayer	Would	Perspektivering; ikke baseline.
Flere biomes/variationer	Would	Perspektivering; ikke baseline.
Flere dyr og crop-varianter	Would	Perspektivering; ikke baseline.
Avanceret byggesystem	Would	Perspektivering; ud over “basic”.

3.6 Acceptkriterier

Kravene anses for opfyldt når følgende kan verificeres i en testsession:

Farming: Spilleren kan plante, afgrøden ændrer vækststadie via time/day-night, og høst tilføjer korrekt item til inventory samt opdaterer UI.

Ressourcesamling: Interaktion med en ressource i verdenen tilføjer korrekt item/mængde til inventory og opdaterer UI uden fejl.

Exploration: Spilleren kan bevæge sig og interagere med verdenens elementer i mindst ét område uden fejl.

Inventory: Inventory kan åbnes/lukkes, viser korrekt indhold, og opdateres korrekt ved harvest/pickup.

Day/Night: Systemet skifter mellem mindst to faser (dag/nat) og påvirker spillets tilstand (fx visuel tilstand og/eller crop-vækst).

Basic building/placement: Spilleren kan placere mindst én type objekt; objektet vises korrekt og kan ikke placeres på ugyldige steder (afgrænses i implementering).

Stabilitet: Spillet kan køre i minimum 30 minutter ved normal brug uden crash.

Ydelse: Spillet opleves stabilt ved normal brug på projektets via brug af almen pc.

---

4. Eksterne grænseflader

4.1 Brugergrænseflade

Spillet styres via tastatur og mus. UI skal vise inventory og ressourcer.

4.2 Hardware

PC med Windows samt mus og tastatur.
Windows-version er 10 eller 11.

4.3 Software

Unity Engine og Windows OS.

---

5. Krav til ydelse

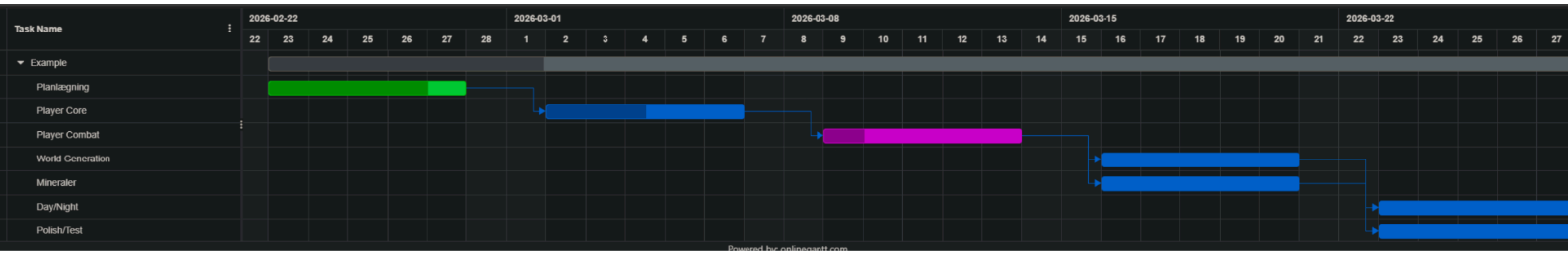
- Spillet skal køre stabilt ved 60 FPS på testmaskinen ved normal brug.
- Input fra spilleren må ikke føles forsinket ved normal brug.

---

6. Kvalitetsfaktorer

Spillet må ikke crashe ved normal gameplay (pålidelighed). Systemer skal være modulære (vedligeholdelse). Nye gameplay-systemer skal kunne tilføjes senere (udvidelsesvenlighed). Spilleren skal kunne forstå spillet uden tutorial (brugervenlighed).

Tidsplan – AKR H5 Projekt



Download: [Tidsplan PDF]